

# Extending support for class types as non-type template parameters

Document #: P2484R0  
Date: 2021-11-16  
Project: Programming Language C++  
Audience: EWG  
Reply-to: Richard Smith  
<[richard@metafoo.co.uk](mailto:richard@metafoo.co.uk)>  
Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## Contents

---

- [1 Abstract](#)
- [2 Introduction](#)
- [3 Proposal: `operator template\(\)`](#)
  - [3.1 Use of `operator template`](#)
  - [3.2 Variable-length representation](#)
  - [3.3 `string\_view` and `span`](#)
  - [3.4 Reference types](#)
  - [3.5 Direction for C++23](#)
- [4 Proposal](#)
  - [4.1 Language Wording](#)
  - [4.2 Library Wording](#)
  - [4.3 Feature-test macros](#)
- [5 References](#)

## § 1 Abstract

---

C++20 introduced the ability to have class types as non-type template parameters. This paper extends the set of types that can be used as non-type template parameters (to allow `std::tuple<T...>`, `std::optional<T>`, and `std::variant<T...>`) and provides a direction for extending it further in the future (to eventually allow `std::vector<T>` and `std::string`).

## § 2 Introduction

---

[[P0732R2](#)] first introduced the ability to have class types as non-type template parameters. The original design was based on defaulting `operator<=>`. But there were problems with and limits to this approach, as described in [[P1907R0](#)]. A subsequent design, [[P1907R1](#)], was adopted for C++20.

This design introduces the term *structural type*, as defined in 13.2 [[temp.param](#)]/7:

7 A *structural type* is one of the following:

- (7.1) — a scalar type, or
- (7.2) — an lvalue reference type, or
- (7.3) — a literal class type with the following properties:
  - (7.3.1) — all base classes and non-static data members are public and non-mutable and
  - (7.3.2) — the types of all bases classes and non-static data members are structural types or (possibly multi-dimensional) array thereof.

The all-public restriction is to ensure that doing template equivalence on every member is a sensible decision to make, as the kind of type for which this is wrong (e.g. `std::vector<int>`) will likely have its members private.

The result of this is that many types become usable as non-type template parameters, like `std::pair<int, int>` and `std::array<int, 2>`. But many other similar ones don't, like `std::tuple<int, int>` or `std::optional<int>`. For both of these, member-wise equivalence would actually do the right thing - but these types are not going to be implemented with all-public members, so they just don't work with the C++20 rules. All we need for `tuple` and `optional` and `variant` is the ability to opt in to the default member-wise equivalence rules we already have.

But going forward, that's not quite sufficient for several important types. Eventually, it would be nice to be able to use `std::vector<T>` and `std::string` as non-type template parameters. A `string` might be implemented as `tuple<char*, char*, char*>` (or perhaps one pointer and two sizes), but examining all three pointer values is not the right model, otherwise code like this would never work:

```
template <std::string S> struct X { };  
  
X<"hello"> a;  
X<"hello"> b;  
a = b;
```

The expectation is that `a` and `b` have the same type, but if template equivalence were based on the underlying pointers of the string, those two strings would have allocated their memory differently and so would have different pointers! We need something different here.

## § 3 Proposal: `operator template()`

The proposal is that a type, `T`, can define an `operator template` which returns a type `R`. `R` must be a structural type, and acts as the representation of `T`. `T` must also be constructible from `R`.

For example:

```
class A {  
private:  
    int i;  
  
    struct Repr { int i; };  
};
```

```

constexpr A(Repr r) : i(r.i) { }
constexpr auto operator template() const -> Repr { return {i}; }
public:
    constexpr A(int i) : i(i) { }
};

template <A a> struct X { };

```

A by default is not structural (it has a private member), but its `operator template` returns `Repr` (which is structural) and A is constructible from `Repr`. The compiler will use `Repr` to determine A's template equivalence rules (as well as its mangling).

`A{1}` and `A{1}` are equivalent because `A::Repr{1}` and `A::Repr{1}` are equivalent.

This example can be simplified. We need some representation that can encapsulate an `int`, but we don't need a whole new type for that:

```

class A2 {
private:
    int i;

    constexpr auto operator template() const -> int { return i; }
public:
    constexpr A2(int i) : i(i) { }
};

template <A2 a> struct X { };

```

The above implementation is also sufficient, because `int` is structural.

But extending the above example to handle *multiple* private members would be very tedious if we had to do it by hand. What would you do for `tuple`? Implement a whole new `tuple` that is all-public instead of all-private? So instead, the model allows for defaulting `operator template()`:

```

class A3 {
private:
    int i;
    int j;

    constexpr auto operator template() const = default;
public:
    constexpr A3(int i, int j) : i(i), j(j) { }
};

template <A3 a> struct X { };

```

This paper is proposing the above be valid. A type with a defaulted `operator template` would base its equivalence on all of its base classes and subobjects, same as a C++20 structural class type. The only difference would be that those base classes and subobjects would be allowed to be private.

Note, though, that this is not recursive:

```

class B {
    int i;
};

class D : B {
    int j;
    constexpr auto operator template() const = default;
};

template <D d> // error: D is not structural because B is not structural
struct Y { }; // ... and B is not structural because it has a private
               member

```

## § 3.1 Use of operator template

The only intent of `operator template` is to allow the compiler to determine template equivalence and mangling. No program need ever invoke it for any reason, so no provisions need to be made in the language for allowing it or defining what that means. This avoids the question of what exactly the return type of a defaulted `operator template` is: it doesn't matter, it's just annotation.

No program ever needs to invoke an `operator template` because of the recursive nature of the definition of structural. In order to incorporate some user-defined type `C` into your mangling, you simply use it directly:

```

class A {
private:
    C c; // some user-defined type (possibly C++20 structural, possibly has
         operator template)
    D d; // some other user-defined type that doesn't participate in
         mangling for some reason

    struct Repr { C c; };
    constexpr auto operator template() const { return Repr{c}; }
    explicit constexpr A(Repr);
};

```

If `C` is structural, regardless of how it gets there (whether `C` is an alias for `int` or has a custom `operator template`), `Repr` is structural and correctly uses `C`'s equivalence rules. No need for any `operator template` invocation here.

## § 3.2 Variable-length representation

The model for defaulted `operator template` allows for letting `tuple`, `optional`, and `variant` opt in to being used as non-type template parameters. The simple member-wise equivalence is correct for all of these types. But it doesn't help us with `vector` or `string`. For that, we need some kind of variable-length type that the compiler recognizes as defining a representation.

The obvious choice there would be: `vector<T>`.

That is, `vector<int>` would just be usable by default (as being a vector of a structural type, `int`), while `string` would opt in by doing:

```

class simplified_string {
    char* begin_;
    char* end_;
    char* capacity_;

    struct repr { std::vector<char> v; };
    constexpr simplified_string(repr r)
        : simplified_string(r.v.begin(), r.v.end())
    { }

    constexpr auto operator template() const -> repr {
        return repr{.v=std::vector(begin_, end_)};
    }
};

```

However, in order to support this approach, the language needs to be able to support non-transient constexpr allocation. Otherwise, non-type template parameters of string or vector type can't even exist. [\[P0784R7\]](#) originally attempted to solve this problem by introducing `std::mark_immutable_if_constexpr`, but this direction was rejected. [\[P1974R0\]](#) proposes to solve this problem using a new `propconst` qualifier.

Regardless of which approach is taken, once the language supports non-transient constexpr allocation, the `operator template` model can be extended to recognize `vector<T>` as being a structural type when `T` is a structural type.

## § 3.3 string\_view and span

While `string_view` compares the contents that it refers to and `span` should as well, the question is: how should these types behave were they to be allowed as non-type template parameters? Put differently:

```

const char a[] = "Hello";
const char b[] = "Hello";
template <string_view S> struct C { };

```

Are `C<a>` and `C<b>` the same type (because `string_view(a) == string_view(b)`) or different types (because their pointers point to different storage)? It basically has *has* to be the latter interpretation. Template equivalence is not `==`, which is why we replaced P0732 with P1907 to begin with. Users that want the former interpretation will have to use `string`, not `string_view`.

This begs the question of whether `string_view` and `span` should be usable as non-type template parameters (i.e. by providing a defaulted `operator template`), but this paper takes no position on that question.

## § 3.4 Reference types

An earlier example in this paper illustrated a custom `operator template` returning an `int`. It is worth considering what would happen if it were instead written, accidentally, to return an `int&`:

```

class A4 {
private:

```

```

    int i;

    constexpr auto operator template() const -> int const& { return i; }
public:
    constexpr A4(int i) : i(i) { }
};

template <A4 a> struct X { };
constexpr A4 i = 1;
constexpr A4 j = 1;

```

`int const&` is also a structural type (lvalue references are structural), but it is *differently* structural from `int`. Equivalence for `int const&` is based on the *address*, while for `int` it's based on the value. `X<i>` and `X<j>` would have to be different types, because their underlying `ints` are different. This mistake would be pretty broken.

However, the compiler should be able to reject such cases, because once we create `X<i>`, the representation of the template parameter object would be different from the representation of `i`.

But there will be cases where it is the correct behavior to return a reference type from `operator template`, so it's not something that can be rejected out of hand.

## § 3.5 Direction for C++23

Because we don't have non-transient `constexpr` allocation yet, the only really interesting cases for `operator template` are those that let you use types with private members as non-type template parameters. So while this model presents a clear direction for how to extend support in the future to allow vector, string, and others to be usable as non-type template parameters, the C++23 paper is a lot narrower: only allow defaulted `operator template`.

This direction allows tuple, optional, and variant, and lots of other class types. Which seems plenty useful.

As discussed earlier, we also for now say that `operator template` *cannot* be invoked by the program - it's *solely* for use by the compiler. This avoids the question of what happens if a program refers to it and what return type they see: there simply will be no such reference. There can be only one `operator template` per class, its *cv-qualifier-seq* must be `const` and its *ref-qualifier* must be empty. Perhaps in the future, these restrictions can be lifted if the need arises, but being conservative here doesn't deprive us of functionality.

## § 4 Proposal

A class type can define `operator template` as defaulted (returning `auto`, with a *cv-qualifier-seq* of `const`, and no *ref-qualifier*) in the body of the class. A class type with such an `operator template` is a structural type if all of its base classes and non-static data members have structural type and none of them are `mutable`.

This is only valid if all base classes and non-static data members have structural types - however we don't want to call this ill-formed if this rule is violated. If `tuple<int, non_structural>` providing a defaulted `operator template` were ill-formed, then `tuple` would have to constrain its `operator template` on all the types being structural, but that's basically the only constraint that's ever meaningful - so it seems reasonable to have defaulting `operator template` actually mean that. But even a (non-template) class having a string member defining `operator template` as defaulted doesn't worth rejecting, for the same reasons as laid out in [P2448R0]: string will eventually be usable as a non-type template parameter, so let users write the declaration early.

Add defaulted `operator template` to `std::tuple`, `std::optional`, and `std::variant`.

## § 4.1 Language Wording

Extend 6.1 [basic.pre]:

- 4 A *name* is an *identifier* ([lex.name]), *operator-function-id* ([over.oper]), *literal-operator-id* ([over.literal]), *template-representation-function-id* ([class.conv.template]), or *conversion-function-id* ([class.conv.fct]).
- 9 Two names are the same if
  - [...]
  - (9.3) — they are *conversion-function-ids* formed with equivalent ([temp.over.link]) types, or
  - (9.3\*) — they are both *template-representation-function-ids* ([class.conv.template]), or
  - (9.4) — they are *literal-operator-ids* ([over.literal]) formed with the same literal suffix identifier.

Add to the grammar of *unqualified-id* in 7.5.4.2 [expr.prim.id.unqual]:

```
unqualified-id:
    identifier
    operator-function-id
    conversion-function-id
+   template-representation-function-id
    literal-operator-id
    ~ type-name
    ~ decltype-specifier
    template-id
```

- 1 An *identifier* is only an *id-expression* if it has been suitably declared ([dcl.dcl]) or if it appears as part of a *declarator-id* ([dcl.decl]). An identifier that names a coroutine parameter refers to the copy of the parameter ([dcl.fct.def.coroutine]). A *template-representation-function-id* shall only appear as part of a *declarator-id*.

[Note 1: For *operator-function-ids*, see [over.oper]; for *conversion-function-ids*, see [class.conv.fct]; for *template-representation-function-ids*, see [class.conv.template]; for *literal-operator-ids*, see [over.literal]; for *template-ids*, see [temp.names]. ... — end note]

Add a new clause called “Template representation functions” after 11.4.8.3 [\[class.conv.fct\]](#) that will define `operator template`:

- 2 A member function with a name of the form:

```
template-representation-function-id:  
operator template
```

shall have no parameters, have a *cv-qualifier-seq* consisting of exactly `const`, have no *ref-qualifier*, have a return type of `auto`, and shall be defined as defaulted on its first declaration. Such a function is called a *template representation function*. [Note: A template representation function can be used to opt a class type with private data members or private base classes into being a structural type ([temp.param]). -end note]

Change 13.2 [\[temp.param\]](#)/7:

- 7 A *structural type* is one of the following:

- (7.1) — a scalar type, or
- (7.2) — an lvalue reference type, or
- (7.3) — a literal class type with the following properties:
  - (7.3.1) — `no direct or indirect subobject is mutable` and
  - (7.3.2) — `either the class defines a template representation function ([??]) or` all base classes and non-static data members are public ~~and non-mutable~~ and
  - (7.3.3) — the types of all bases classes and non-static data members are structural types or (possibly multi-dimensional) array thereof.

No changes to 13.6 [\[temp.type\]](#) necessary, since the class type equivalence rule (“their corresponding direct subobjects and reference members are template-argument-equivalent”) is still preserved with this change.

Add a note to 13.9.3 [\[temp.explicit\]](#):

- 12 An explicit instantiation of a prospective destructor ([class.dtor]) shall correspond to the selected destructor of the class.

[Note: an explicit instantiation of a *template-representation-function-id* is not allowed. -end note]

## § 4.2 Library Wording

Add to 20.5.3 [\[tuple.tuple\]](#) the wording we have for structurality. We don’t provide a defaulted `operator template` here, since implementers can achieve this however they want (maybe by making everything public?). The important thing is that we define that it must work and what it means:

- 1 `tuple<Types...>` is a structural type ([temp.param]) if every Type in `Types...` is a structural type. Two values `t1` and `t2` of type `tuple<Types...>` are template-argument-equivalent ([temp.type]) if and only if each pair of corresponding elements from `t1` and `t2` are template-argument-equivalent.

Add similar to 20.6.3.1 [\[optional.optional.general\]](#):

- 2 Member `val` is provided for exposition only. When an `optional<T>` object contains a value, `val` points to the contained value.
- 3  $\tau$  shall be a type other than `cv in_place_t` or `cv nullopt_t` that meets the *Cpp17Destructible* requirements (Table 34).
- 4 `optional<T>` is a structural type ([temp.param]) if  $\tau$  is a structural type. Two values `o1` and `o2` of type `optional<T>` are template-argument-equivalent ([temp.type]) if and only if either neither `o1` nor `o2` contain a value or if both contain a value and `*o1` and `*o2` are template-argument-equivalent

And similar to 20.7.3.1 [\[variant.variant.general\]](#):

- 2 All types in `Types` shall meet the *Cpp17Destructible* requirements (Table 34).
- 3 A program that instantiates the definition of `variant` with no template arguments is ill-formed.
- 4 `variant<Types...>` is a structural type ([temp.param]) if every Type in `Types...` is a structural type. Two values `v1` and `v2` of type `variant<Types...>` are template-argument-equivalent ([temp.type]) if and only if both `v1` and `v2` hold a value, `v1.index() == v2.index()`, and `get<v1.index()>(v1)` and `get<v2.index()>(v2)` are template-argument-equivalent.

## § 4.3 Feature-test macros

Bump the non-type template argument macro in 15.11 [\[cpp.predefined\]](#):

```
__cpp_nontype_template_args 201911L 2022XXL
```

Bump the corresponding library feature test macros in 17.3.2 [\[version.syn\]](#). These seem like the most appropriate choices:

```
#define __cpp_lib_constexpr_tuple 201811L 2022XXL // also in <tuple>
#define __cpp_lib_optional 202106L 2022XXL // also in <optional>
#define __cpp_lib_variant 202106L 2022XXL // also in <variant>
```

## § 5 References

[P0732R2] Jeff Snyder, Louis Dionne. 2018-06-06. Class Types in Non-Type Template Parameters.  
<https://wg21.link/p0732r2>

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.

<https://wg21.link/p0784r7>

[P1907R0] Jens Maurer. 2019-10-07. Inconsistencies with non-type template parameters.

<https://wg21.link/p1907r0>

[P1907R1] Jens Maurer. 2019-11-08. Inconsistencies with non-type template parameters.

<https://wg21.link/p1907r1>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.

<https://wg21.link/p1974r0>

[P2448R0] Barry Revzin. 2021-10-14. Relaxing some constexpr restrictions.

<https://wg21.link/p2448r0>